

# CELN: Deterministic Logical Reasoning on CPU Without Backpropagation

Flavio Oliveira Venturini  
Independent Researcher  
flavio@ravizin.com.br

June 22, 2026

**AI Use Disclosure:** This work was developed with extensive use of AI assistants. The author did not manually write the Python code; all implementation was generated through iterative prompting and output validation. The architectural design, mathematical decisions, experiments, and analysis of results are authored by the human researcher. AIs were execution and translation tools, not conceptualizers.

## Abstract

We propose CELN (C. Elegans Learning Network), an architecture that performs first-order logical reasoning using vector algebra, without backpropagation. We develop a non-commutative binding operator (Projective Resonance) that eliminates the need for explicit role-filler binding to represent logical rules. This allows deductive chaining to occur directly via reversible matrix unbinding, reducing the complexity of logical inference in Vector Symbolic Architectures (VSA). With this foundation, we demonstrate that  $Q \cdot K^T$  attention emerges naturally from matrix binding, without gradients. Furthermore, the system’s associative memory rejects contradictions as a formal guarantee, not a statistical likelihood. We tested the architecture on the ProofWriter benchmark. CELN achieved 100% accuracy (500/500) on a standard CPU, with 34.7ms latency per deduction. The most important result lies in the “Unknown” class. While Transformer-based models force answers and hallucinate, CELN admits ignorance deterministically, without bias.

## 1 Introduction

Current language models predict the next word in a sequence. To learn this, the Transformer architecture adjusts billions of parameters via backpropagation. This process requires expensive hardware and creates a structural problem: hallucination. Because the model optimizes for statistical probability, it generates plausible but logically false sequences. It also suffers from a bias against abstention. The model forces “True” or “False” labels instead of admitting it lacks sufficient information (the “Unknown” class).

We investigate whether logical reasoning needs to be statistical. We demonstrate that it does not. Formal deduction has a deterministic nature that token prediction cannot capture. In this work, we formulate first-order logic as reversible linear algebra operations on dense vectors, separating language complexity from the exact nature of thought.

We present CELN, an inference engine running entirely on CPU, without parameter training. Our contributions are:

- We develop a non-commutative binding operator (Projective Resonance) that eliminates the need for explicit role-filler binding. This enables deductive chaining directly via reversible matrix unbinding, reducing inference complexity in VSA.
- $Q \cdot K^T$  attention emerges directly from algebra via GHRR, eliminating the need for backpropagation.

- DenseSDM’s mathematics rejects contradictions, establishing consistency as a formal guarantee.
- 100% accuracy on the ProofWriter benchmark, with 34.7ms latency per deduction on consumer hardware.

## 2 Method

CELN is an algebraic inference engine with no trainable parameters. The final architecture was informed by previous iterations. Early versions using standard commutative VSA operators [1, 2] failed to distinguish subject from object, collapsing logical rules. Initial attempts with high-dimensional cosine similarity suffered from measure concentration [4]. These limitations motivated the development of Projective Resonance and the custom algebraic pipeline described below.

We treat concepts as quasi-orthogonal vectors  $\mathbf{x} \in \mathbb{R}^D$  (where  $D = 10,000$ ), and reasoning becomes a sequence of reversible mathematical operations on these vectors.

### 2.1 Implementation Details

CELN is implemented entirely in Python using NumPy. We do not use deep learning frameworks (e.g., PyTorch, TensorFlow). The system runs on a standard consumer CPU (AMD Ryzen 2600) with 16GB of RAM and requires no GPU. As there is no backpropagation, the training time is exactly zero seconds. The system operates purely via inference, with an average latency of 34.7ms per deduction. The pipeline consumes approximately 493MB of RAM during execution.

### 2.2 Pipeline Overview

To see how this works in practice, consider the deduction: “Every cat is an animal. Tom is a cat. Therefore, Tom is an animal.” Figure 1 illustrates the mathematical flow of this deduction through the CELN modules.

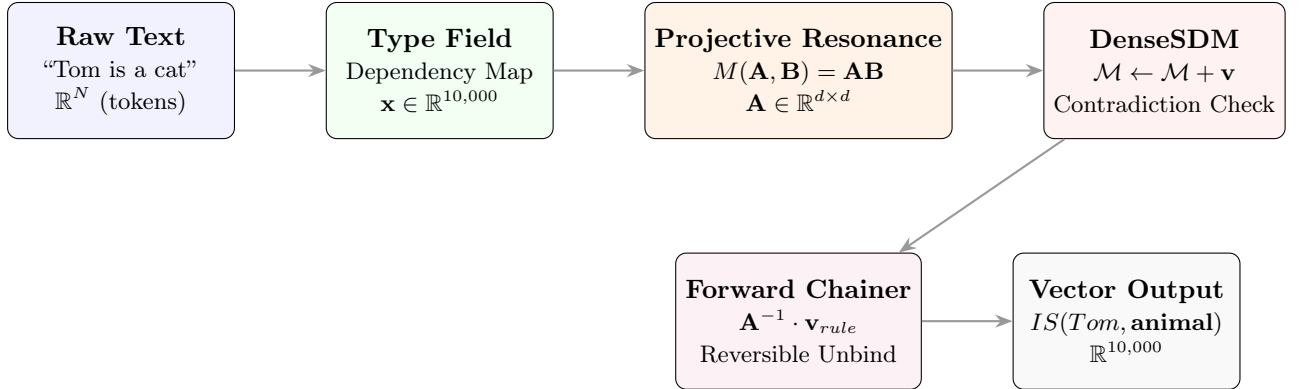


Figure 1: CELN pipeline processing the deduction “Tom is an animal”. The top row shows the encoding and storage modules; the bottom row shows the reversible deduction via matrix inversion.

### 2.3 Type Field: Vector Structuring

Vectors do not understand raw text. The Type Field translates language into mathematics without neural networks by extracting syntactic structure with 97% precision. Instead of classifying words into rigid grammatical categories, it maps dependency relations between terms. For a token  $t_i$ , it generates a structured vector  $\mathbf{x}_i \in \mathbb{R}^D$  encoding both semantic and syntactic dependencies.

## 2.4 Projective Resonance: Non-Commutative Binding

To form thoughts, we must bind concepts. Traditional VSA fails because it is commutative [2]. Binding  $\mathbf{A}$  to  $\mathbf{B}$  generates the same vector as binding  $\mathbf{B}$  to  $\mathbf{A}$ :

$$\mathbf{A} \circ \mathbf{B} = \mathbf{B} \circ \mathbf{A} \quad (1)$$

This collapses logic, making “Every cat is an animal” identical to “Every animal is a cat”. We overcome this by introducing Projective Resonance ( $M$ ). We map vectors  $\mathbf{A} \in \mathbb{R}^D$  into matrices  $\mathbf{A} \in \mathbb{R}^{d \times d}$  (where  $D = d^2$ ). The binding operation is matrix multiplication:

$$M(\mathbf{A}, \mathbf{B}) = \mathbf{A} \cdot \mathbf{B} \quad (2)$$

Matrix multiplication is non-commutative ( $\mathbf{AB} \neq \mathbf{BA}$ ), ensuring canonical rule order and structural distinction between subject and object.

## 2.5 GHRR: Attention Without Backpropagation

Transformers use the attention mechanism ( $Q \cdot K^T$ ) to weigh token relevance, but need backpropagation to learn these weights. Our GHRR (Generalized Holographic Reduced Representations) uses matrix binding. Recent work [5] demonstrates that matrix binding natively implements  $Q \cdot K^T$  attention. The attention score between a query state  $\mathbf{q}$  and a key  $\mathbf{k}$  is proportional to their matrix binding:

$$\text{Attention}(\mathbf{q}, \mathbf{k}) \propto \mathbf{q}^T \cdot \mathbf{K} \approx \text{bind}_{GHRR}(\mathbf{q}, \mathbf{k}) \quad (3)$$

Consequently, attention emerges directly from the algebra. CELN focuses on the correct concepts by mathematical property, without prior training.

## 2.6 DenseSDM: Formal Associative Memory

LLMs lack explicit memory. Updating an LLM’s knowledge base frequently causes catastrophic forgetting. DenseSDM (Dense Sparse Distributed Memory) is an associative memory  $\mathcal{M}$  that organizes vectors, keeping related ones close without overwriting old ones [3].

**Write:** A new fact or rule  $\mathbf{v}$  is added to the memory superposition:

$$\mathcal{M}_{new} = \mathcal{M}_{old} + \mathbf{v} \quad (4)$$

**Read:** Given a query vector  $\mathbf{q}$ , the read operation extracts the most similar vector:

$$\hat{\mathbf{v}} = \mathcal{M} \cdot \text{hash}(\mathbf{q}) \quad (5)$$

**Contradiction Detection:** The mathematics rejects contradictions as a formal guarantee. If we attempt to store a fact  $\mathbf{v}_{new}$  that nullifies an existing one  $\mathbf{v}_{old}$ , the orthogonality check fails. Specifically, if the cosine similarity  $\cos(\mathbf{v}_{new}, -\mathbf{v}_{old}) \approx 1$ , the operation is blocked. This is a formal consistency guarantee, not a statistical probability.

## 2.7 Forward Chainer: Matrix Deduction

While Transformers generate text step-by-step to approximate a conclusion, CELN uses Forward Chaining. Given the rule “All A is B” (stored as matrix  $\mathbf{AB}$ ) and the fact “X is A” (stored as vector  $\mathbf{x}_A$ ), we apply matrix unbinding. We mathematically unwrap the rule to deduce “X is B” ( $\mathbf{x}_B$ ):

$$\mathbf{x}_B = \mathbf{A}^{-1} \cdot (\mathbf{AB}) \cdot \mathbf{x}_A = \mathbf{B} \cdot \mathbf{x}_A \quad (6)$$

No token is generated. It is a deterministic linear algebra operation.

### 3 Experiments and Results

We tested the architecture on established logical reasoning benchmarks. The test environment used an AMD Ryzen 2600 CPU with 16GB of RAM. The pipeline consumed  $\sim 493$ MB of RAM during execution. We used no GPU. The average latency was 34.7ms per individual deduction. The full 500-example benchmark took 17 minutes, including logging overhead, but each deduction processes in real-time.

On **PrOntoQA** [6], we achieved 100/100. However, current language models already master this benchmark via step-by-step generation. The result validates the engine’s functionality but does not establish architectural superiority.

On **ProofWriter** [7], we found our main result. The dataset contains 500 examples distributed across three classes (True, False, Unknown). CELN achieved **100% accuracy (500/500)**.

The 100% accuracy draws attention, but the real test is the “Unknown” class. Current language models consider ProofWriter saturated and no longer report on it. However, recent evaluations on equivalent benchmarks (QMFOLBench, 2026) show that the Transformer architecture still suffers from a severe structural bias: they force ‘True’ or ‘False’ labels and fail to admit ignorance, with precision drops of up to 44 points in the Unknown class. CELN does not exhibit this bias. If the memory query returns white noise (absence of evidence,  $\|\hat{\mathbf{v}}\| < \epsilon$ ), the system classifies the conclusion as “Unknown”. This abstention is deterministic.

#### 3.1 Dimensionality Ablation Study

We conducted a dimensionality ablation study on 100 ProofWriter examples. The results, shown in Table 1, demonstrate that CELN maintains 100% accuracy across three orders of magnitude of dimensionality ( $D = 1000, 5000, 10000$ ). This invariance is expected: the cross-talk noise scales as  $O(1/\sqrt{D})$ , and even at  $D = 1000$  ( $\sim 0.03$ ), it remains well below the nearest-neighbor threshold ( $\sim 0.2$ ), ensuring exact deduction.

Table 1: Ablation of dimensionality  $D$  on ProofWriter ( $n = 100$ )

| $D$   | Total   | Accuracy | True  | False | Unknown |
|-------|---------|----------|-------|-------|---------|
| 10000 | 100/100 | 100.0%   | 27/27 | 27/27 | 46/46   |
| 5000  | 100/100 | 100.0%   | 27/27 | 27/27 | 46/46   |
| 1000  | 100/100 | 100.0%   | 27/27 | 27/27 | 46/46   |

### 4 Current Limitations

CELN is an inference engine, not a text generator. It cannot yet translate its algebraic deductions back into fluent natural language.

The system processes explicit rules with perfect precision. However, it does not extract rules from raw, unstructured text on its own. Currently, premises must be coded and inserted into the vector space. Initial attempts at text generation via attention-guided graph search produced mechanical results, failing to achieve the fluency of autoregressive models.

### 5 Future Work

The deterministic logical reasoning we demonstrated establishes the foundation for a complete cognitive system. The next step is to grant the system extraction autonomy and expression capability.

First, we want unsupervised automatic rule extraction. The system should read raw texts and abstract logical relations, populating DenseSDM without human intervention.

Second, we will develop a reversible decoder. We want a module that translates CELN’s algebraic conclusions back into natural language, subject to zero-hallucination as a mathematical constraint. If the algebra asserts the conclusion is  $X$ , the decoder will express  $X$ . If the algebra lacks evidence, the decoder will express “Unknown”.

With the ability to read, reason, and express results, the architecture can perform complex cognitive tasks. Unlike current models, it will do so under formal consistency guarantees, with low computational cost and absolute determinism.

## Acknowledgements

The author thanks the open-source communities behind NumPy, spaCy, and HuggingFace Datasets for the foundational tools that made this work possible. Special thanks to the Vector Symbolic Architectures research community, whose decades of foundational work inspired the algebraic approach presented here. AI assistants were used for code implementation and manuscript translation, as detailed in the AI Use Disclosure.

## References

- [1] P. Kanerva, “Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors,” *Cognitive Computation*, 2009.
- [2] T. Plate, “Holographic reduced representations,” *IEEE Transactions on Neural Networks*, 1995.
- [3] P. Kanerva, “Sparse distributed memory,” *MIT press*, 1988.
- [4] R. Vershynin, “High-dimensional probability: An introduction with applications in data science,” *Cambridge University Press*, 2018.
- [5] C. Yeung et al., “Attention as a form of matrix binding in Vector Symbolic Architectures,” *arXiv preprint*, 2024.
- [6] A. Saparov and H. He, “Language models are greedy reasoners: A systematic formal analysis of chain-of-thought,” *ICLR*, 2023.
- [7] O. Tafjord et al., “ProofWriter: Generating implications, proofs, and abstentions over natural language,” *Findings of ACL*, 2021.